

Saving Arduino Data Via Serial Port

Author: Brendan Klem

Goal: Send the data via serial to the computer, and list the data in a specific format.

Materials List

- Arduino Uno R3
- Resistors: 10kΩ, 1MΩ
- 5 Wires
- Pressure sensor
- Vibration sensor
- USB-C Cable

General Notes

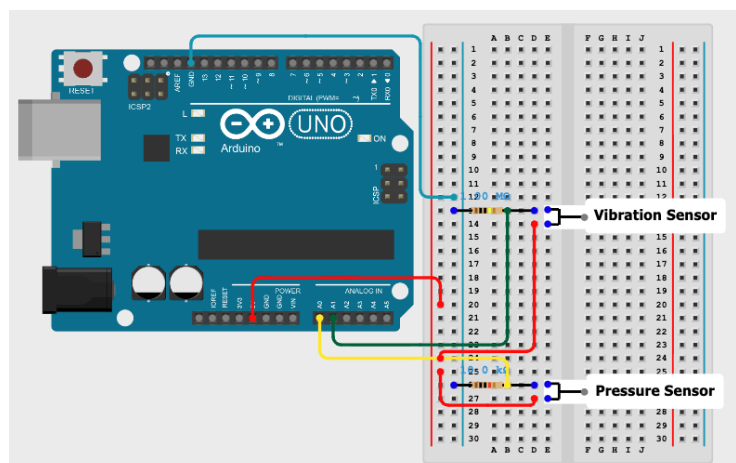
There were some issues related to the arduino that I had to overcome. Some of them include:

- Arduino has a default timer that goes to the millisecond, but to my knowledge it is not possible to transfer that data to python, so I added the `millis()` function to keep track of the time.
- I tested more resistor values and came to the conclusion that the 10kΩ and 1MΩ resistors work best for the respective sensors.
- I did more testing with the vibration sensor, and I seem to have gotten more consistent outputs, so I decided to combine them both into one circuit to finally test both values and save the results to a file.

When looking at the data, it might not seem like it outputs data every millisecond, but that is just due to the fact that I input `delay(100);`, which delays the output for every 100 milliseconds. This can easily be changed to any value so theoretically if there was no delay you would get data from every millisecond, with the downside being a lot more data to parse through.

Circuit Layout:

Here is the circuit layout I used to connect both the pressure sensor and the vibration sensor to the arduino. The IMU shouldn't be too difficult to attach to the circuit as well.



The vibration sensor requires a 1MΩ resistor, while the pressure sensor requires 10kΩ to get the most consistent results.

Arduino Code

The arduino code is listed below and is what displays the output in the format: milliseconds, pressure_value, vibration_value.

```
int pressurePin = A0;
int vibrationPin = A1;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int x = analogRead(pressurePin);
  int y = analogRead(vibrationPin);

  Serial.print(millis());
  Serial.print(", ");
  Serial.print(x);
  Serial.print(", ");
  Serial.println(y);

  delay(100);
}
```

Installing Python Serial Library

Moving the output from the sensors through your computer's serial port isn't possible through arduino alone, so leveraging python is the best solution to get all the data into one file. The first step is installing python from python.org. To get access the serial port we need to install the python serial library, so in terminal enter:

```
pip3 install pyserial
```

Then find the arduino port:

```
ls /dev/cu.*
```

Where the output should look something like:

```
/dev/cu.usbmodemDC7839C837465
```

This is the serial port for your computer. This is what is then entered into python under the variable `ser` so the program knows the correct output port.

Python Code to Save Data to File

This code

```
import serial, os

ser = serial.Serial('/dev/cu.usbmodemDC7839C837465', 115200)

path = os.path.join(os.path.expanduser("~"), "Desktop", "sensor_data.csv")

with open(path, "w") as f:
    print("Saving to:", path)

    f.write("timestamp_ms,force_raw,vibration_raw\n")

    while True:
        line = ser.readline().decode().strip()
        print(line)

        f.write(line + "\n")
        f.flush()
```

As soon as you stop the code, all the data that has been gathered is placed into

```
/Users/YOUR_USERNAME/Desktop/sensor_data.csv
```

Sample Output

When you open the file under Desktop -> sensor_data.csv, you will see:

sensor_data		
timestamp_ms	force_raw	vibration_raw
61904	3	5
62005	3	4
62106	2	4
62207	2	4
62308	3	4
62409	15	3
62510	48	5
62611	79	4
62712	116	4
62813	132	4
62914	136	4
63015	135	4

This has the data formatted as milliseconds, pressure_value, vibration_value. Each row is separated by approximately 100 milliseconds because of `delay(100);` as mentioned in the General Notes. The arduino processes and sends data to the IDE all in about 1 millisecond, which would explain why it is separated by 101 milliseconds instead of just 100 from the delay. Another thing to add is that the timestamps are relative to each, so it is basically just counting up in milliseconds. If you want an absolute clock that captures the real time, that can absolutely be done on an arduino but it requires a hardware real time clock and the software RTC library to get the real time.